

Robotic Process Automation

Laboratory Work No. 3

Developing a Telegram Bot Integrated with a Back-End Application

Student: Matvei Krautsou

Project: Campus Shop — Telegram e-commerce bot and Mini App

Date: 6 October 2026

1. Objective

Develop a Python Telegram bot integrated with a separate back-end application. Demonstrate inline keyboards, stickers, and a Mini App. Implement the advanced task by placing the main business logic and persistent data in the back end.

2. Business area and business process

The selected area is e-commerce: a campus shop selling stationery and study accessories. Its customer process is: open the bot, browse the catalog, select a product, confirm the purchase, receive an order number and total, and view personal order history. A customer can also use the Mini App to choose quantities and create orders.

The server checks product existence and inventory, obtains the authoritative price, calculates the total, reduces the remaining stock, and saves the order. Cancellation makes no database changes. Insufficient stock produces an error and creates no order. This prototype records purchases; it does not charge money or arrange delivery.

3. Technologies and architecture

Python 3.12 implements the Telegram bot with standard-library HTTP requests and long polling. Node.js 24 implements the HTTP back end. Its built-in SQLite driver stores products and orders. HTML, CSS, and JavaScript implement the Mini App. The Telegram WebApp SDK supplies initialization data and success haptic feedback.

Bot interaction: Telegram user -> Telegram Bot API -> Python bot -> Node.js REST API -> SQLite.

Mini App interaction: Telegram user -> HTML/JavaScript Mini App -> Node.js REST API -> SQLite.

The bot handles conversation and presentation. The back end owns stock validation, price calculation, order creation, and history. Both interfaces call the same business API. Runtime application code requires no npm or pip packages. Report generation uses ReportLab, python-docx, and Pillow; these are separate from application runtime dependencies.

4. Bot registration and Telegram interaction

The project bot is @Lab3MKbot; its visible chat name in the supplied screenshots is Lab3_MK. The token was obtained through BotFather and loaded into the local runtime configuration. Telegram getMe successfully verified the token and bot username. The token is excluded from the source repository and report.

The `/start` command sends a welcome message and an inline keyboard containing Catalog and My orders. It then uploads an original static WEBP shopping-bag sticker. The Catalog button returns product buttons containing the name, price, remaining stock, and product ID in callback data. Choosing a product opens Confirm order / Cancel. Confirm order posts an order to the back end; Cancel returns to the catalog.

The bot removes the confirmation keyboard after a successful purchase. Each confirmed bot purchase orders one item. The Mini App supports quantities from 1 to 100. Repeated or retried purchase requests are not protected by an idempotency key and can create separate orders; this remains a prototype limitation.

Open Mini App is displayed in private chats only when `WEB_APP_URL` contains a configured HTTPS address. In the supplied Telegram screenshots this button is absent; therefore those screenshots demonstrate the bot workflow, not Mini App deployment within Telegram.

5. Mini App implementation

The Mini App is served by the same backend at `/`. Its responsive catalog shows product cards, euro prices, inventory counts, quantity inputs, and Order buttons. My orders displays order numbers, quantities, and totals. A successful purchase updates the catalog and displays a confirmation. Backend errors are shown in the status area.

When opened inside Telegram, the page reads `Telegram.WebApp.initData` and sends it in `X-Telegram-Init-Data`. The server authenticates it before allowing purchases or accessing history. For isolated local browser demonstrations, `DEMO_MODE` uses a fixed test identity. This mode must not be used on a public deployment.

The four screenshots in Section 10 were captured from the actual page using Chromium at a mobile viewport of 430 x 932 pixels. They use a separate in-memory demo database and do not alter the live bot's database. They are genuine local-browser screenshots, not generated illustrations and not screenshots of a deployed Telegram WebView.

6. Back-end API

`GET /health` returns service readiness. `GET /api/products` returns the catalog. `POST /api/orders` accepts `product_id` and quantity and returns the stored order. `GET /api/orders` returns only the authenticated user's orders. `GET /` serves the Mini App.

The server rejects invalid JSON, oversized request bodies, invalid quantities, missing products, unauthenticated requests, and insufficient stock. Relevant response statuses are 200 for retrieval, 201 for creation, 400 for invalid input, 401 for missing authentication, 404 for missing resources, 409 for insufficient stock, and 413 for excessive request size.

A client-supplied price is never used. Prices are stored as integer euro cents, which avoids floating-point rounding during total calculation. The seeded catalog contains Notebook (500 cents), Pen set (300 cents), and Backpack (2500 cents).

7. Database and transaction

The products table contains `id`, `name`, `price`, and `stock`. The orders table contains `id`, `user_id`, `product_id`, `quantity`, `total`, and `created_at`. A foreign key links each order to its product. Database files are stored in `backend/data/` and ignored by Git.

Order creation uses `BEGIN IMMEDIATE`, reads the product, validates stock, subtracts the quantity, inserts the order, and commits. Any failure rolls the transaction back. The stock column also has a non-negative `CHECK` constraint. The initial products are inserted with `INSERT OR IGNORE`, so restarting the application does not overwrite inventory.

8. Authentication and configuration

Python-to-backend requests carry a shared `BACKEND_API_KEY` in `X-API-Key` and specify the Telegram user ID. The Mini App instead supplies signed Telegram initialization data. The back end derives a secret using HMAC-SHA256 with `WebAppData` and the bot token, verifies Telegram's data-check-string signature with a constant-time comparison, checks a maximum age of one hour, and uses the identity from the signed user object.

A user ID supplied in an unsigned Mini App request cannot replace the signed identity. Order-history queries filter by that authenticated identity. Tokens and API keys are loaded from environment variables; `.env` is ignored. HTTPS is required for the public Mini App endpoint.

9. Validation and observed Telegram results

The Node.js integration test passed. Its assertions cover health and HTML responses, three catalog products, unauthorized order rejection, a two-notebook order totalling 1000 cents, invalid quantities, missing products, insufficient stock, unchanged stock after rejection, user-isolated history, a valid Telegram signature, and rejection of tampered initialization data.

Two Python mock tests passed. They verify the `/start` welcome flow, sticker invocation and Mini App keyboard, and that a purchase callback sends the correct backend request and presents the returned confirmation. They exercise Python logic without contacting Telegram.

The real backend health check and authenticated history request returned HTTP 200. Telegram `getMe` verified `@Lab3MKbot`, and `getWebhookInfo` confirmed that the polling bot has no configured webhook.

The student supplied three Telegram screenshots. The first shows `/start`, the welcome message, Catalog / My orders, and the shopping-bag sticker. The second shows the catalog, purchase confirmation, Order #2 confirmed! Total: EUR 5.00, and its history entry. The third shows a backpack order (#3, EUR 25.00), history entries for orders #3 and #2, and updated inventory: Notebook 18, Pen set 30, Backpack 9. Together, these images demonstrate end-to-end bot purchases, personal history, and inventory updates.

The inline Telegram screenshots were visible in the chat but were not exposed as downloadable source files in the build workspace. Their observed contents are documented here; their original images are not embedded in this version. The four Mini App screenshots below are embedded. No Telegram screenshot was fabricated or reconstructed.

10. Mini App screenshots

Figure 1. Product catalog

Actual local browser capture: Notebook EUR 5.00, Pen set EUR 3.00, Backpack EUR 25.00; quantity fields and purchase buttons.

Campus Shop

Study essentials. Order in a few taps.

Notebook

€5.00 · 20 in stock

[Order](#)

Pen set

€3.00 · 30 in stock

[Order](#)

Backpack

€25.00 · 10 in stock

[Order](#)[My orders](#)

Figure 2. Successful purchase

A notebook purchase returns Order #1 and EUR 5.00. Remaining notebook stock changes from 20 to 19.

Campus Shop

Study essentials. Order in a few taps.

Notebook

€5.00 · 19 in stock

[Order](#)

Pen set

€3.00 · 30 in stock

[Order](#)

Backpack

€25.00 · 10 in stock

[Order](#)[My orders](#)

Order #1 confirmed · €5.00

Figure 3. Personal order history

After purchasing a notebook and a backpack, history displays orders #1 and #2 with their respective totals. Order numbers differ from the live Telegram screenshots because this test uses an isolated database.

Campus Shop

Study essentials. Order in a few taps.

Notebook

€5.00 · 19 in stock

[Order](#)

Pen set

€3.00 · 30 in stock

[Order](#)

Backpack

€25.00 · 9 in stock

[Order](#)[My orders](#)

#2 · 1 item(s) · €25.00

#1 · 1 item(s) · €5.00

Figure 4. Insufficient stock

Attempting to order 100 notebooks exceeds available stock. The server returns Not enough stock and does not create a new order.

Campus Shop

Study essentials. Order in a few taps.

Notebook

€5.00 · 19 in stock

Order

Pen set

€3.00 · 30 in stock

Order

Backpack

€25.00 · 9 in stock

OrderMy orders

Not enough stock

11. Reproducing the application

Requirements: Node.js 24+ and Python 3.12+. Copy `.env.example` to a local `.env` and configure `TELEGRAM_BOT_TOKEN`, `BACKEND_API_KEY`, `BACKEND_URL`, and, for the Telegram Mini App, `WEB_APP_URL`. Both processes must receive the same token and API key. The included `welcome.webp` is the original sticker and needs no generation or image library at runtime.

Start the backend from `backend/` with `node server.js`. Start the bot from the project root with `python bot/main.py`. Run a single polling worker per bot token. The bot removes an existing webhook on startup. The backend must be reachable at `BACKEND_URL`.

For local browser testing, start the backend with `DEMO_MODE=1` and open its HTTP root locally. Never enable that setting publicly. Detailed commands and environment handling are provided in `README.md`. Run `npm test` in `backend/` and `python -m unittest -v` in `bot/` to reproduce automated checks.

12. Hosting and limitations

The real backend and bot were started in the current cloud development environment. This does not guarantee 24/7 uptime: stopping or recycling the environment can stop the processes. A permanent deployment must preserve SQLite data and supervise both processes. A public HTTPS endpoint and WEB_APP_URL are still needed to open the Mini App from Telegram; local browser functionality has been verified.

No payment processing, delivery workflow, inventory administration, or production rate limiting is included. API keys, HTTPS termination, backups, and access policies require appropriate configuration for a real deployment. Repeated purchase requests lack idempotency. These limitations do not affect the demonstrated laboratory order and inventory flow.

13. Conclusion

The project implements a Python Telegram shopping bot and a separate Node.js backend with SQLite business logic. It uses inline keyboards, an original sticker, and a responsive Mini App. Automated API and Python tests passed. The supplied Telegram evidence demonstrates live bot purchases, history, and stock changes; real browser screenshots demonstrate the Mini App workflow and validation. The remaining deployment step is permanent hosting with a public HTTPS Mini App URL.

Appendix A. Complete project source

The following source files are reproduced in the generated PDF, DOCX, and HTML report. The GitHub repository additionally contains the original sticker, screenshot images, tests, README, and editable report source.

backend/package.json

```
{"name": "lab3-shop-backend", "version": "1.0.0", "private": true, "type": "module", "engines": {"node": ">=24"}, "scripts": {"start": "node server.js", "test": "node --test"}}
```

backend/server.js

```
import http from 'node:http';
import { DatabaseSync } from 'node:sqlite';
import { createHmac, timingSafeEqual } from 'node:crypto';
import { readFileSync, mkdirSync } from 'node:fs';
import { fileURLToPath } from 'node:url';

export function createApp({dbPath='data/shop.sqlite', token=process.env.TELEGRAM_BOT_TOKEN,
apiKey=process.env.BACKEND_API_KEY, demo=false}={}) {
  if (dbPath !== ':memory:') mkdirSync(fileURLToPath(new URL('./data/', import.meta.url)),
{recursive:true});
  const db = new DatabaseSync(dbPath);
  db.exec('PRAGMA foreign_keys=ON;
  CREATE TABLE IF NOT EXISTS products(id INTEGER PRIMARY KEY, name TEXT NOT NULL, price INTEGER NOT
NULL, stock INTEGER NOT NULL CHECK(stock>=0));
  CREATE TABLE IF NOT EXISTS orders(id INTEGER PRIMARY KEY, user_id TEXT NOT NULL, product_id INTEGER
REFERENCES products(id), quantity INTEGER NOT NULL, total INTEGER NOT NULL, created_at TEXT DEFAULT
CURRENT_TIMESTAMP);
  INSERT OR IGNORE INTO products VALUES(1, 'Notebook', 500, 20), (2, 'Pen
set', 300, 30), (3, 'Backpack', 2500, 10);');
  function identity(req, body) {
    if (apiKey && req.headers['x-api-key'] === apiKey && /\^d+$/\.test(String(body.user_id))) return
String(body.user_id);
    const raw=req.headers['x-telegram-init-data'];
    if(raw && token) {
      const p=new URLSearchParams(raw), hash=p.get('hash'); p.delete('hash');
      const
check=[...p.entries()].sort([(a],[b])=>a.localeCompare(b)).map(([k,v])=>`$k=${v}`).join('\n');
      const secret=createHmac('sha256', 'WebAppData').update(token).digest();
      const expected=createHmac('sha256', secret).update(check).digest();
      if(!hash || !/\^[a-f0-9]{64}$/.test(hash) || !timingSafeEqual(expected, Buffer.from(hash, 'hex')))
throw Error('Unauthorized');
      const age=Math.floor(Date.now()/1000)-Number(p.get('auth_date'));
      if(!Number.isFinite(age) || age<0 || age>3600) throw Error('Unauthorized');
      const user=JSON.parse(p.get('user'))||{};
      if(!Number.isSafeInteger(user.id) || user.id<=0) throw Error('Unauthorized');
      return String(user.id);
    }
    if(demo) return '1';
    throw Error('Unauthorized');
  }
  const server=http.createServer(async(req, res)=>{
    const send=(status, data)=>{res.writeHead(status, {'Content-
Type': 'application/json'}); res.end(JSON.stringify(data));};
    try {
      const path=new URL(req.url, 'http://localhost').pathname;
      if(req.method==='GET' && path=== '/') {res.writeHead(200, {'Content-Type': 'text/html;
charset=utf-8'}); res.end(readFileSync(new URL('./index.html', import.meta.url))); return;}
      if(req.method==='GET' && path=== '/health') return send(200, {status: 'ok'});
      if(req.method==='GET' && path=== '/api/products') return send(200, db.prepare('SELECT * FROM products
ORDER BY id').all());
      let body={};
      if(req.method==='POST') {
        let raw=''; for await(const chunk of req) {raw+=chunk; if(Buffer.byteLength(raw)>16384) return
send(413, {error: 'Request too large'});}
        try {body=JSON.parse(raw);} catch {return send(400, {error: 'Invalid JSON'});}
        if(!body || typeof body!=='object' || Array.isArray(body)) return send(400, {error: 'Invalid
body'});
      }
      if(path=== '/api/orders' && ['GET', 'POST'].includes(req.method)) {
        if(req.method==='GET') body.user_id=new
URL(req.url, 'http://localhost').searchParams.get('user_id');
        const user=identity(req, body);
        if(req.method==='GET') return send(200, db.prepare('SELECT * FROM orders WHERE user_id=? ORDER BY
id DESC').all(user));
        const {product_id, quantity}=body;
        if(!Number.isSafeInteger(product_id)||!Number.isSafeInteger(quantity)||quantity<1||quantity>100)
return send(400, {error: 'Choose a valid product and quantity (1-100)'});
        db.exec('BEGIN IMMEDIATE');
        try {
          const p=db.prepare('SELECT * FROM products WHERE id=?').get(product_id);
          if(!p) {db.exec('ROLLBACK'); return send(404, {error: 'Product not found'});}
          if(p.stock<quantity) {db.exec('ROLLBACK'); return send(409, {error: 'Not enough stock'});}
          db.prepare('UPDATE products SET stock=stock-? WHERE id=?').run(quantity, product_id);
          const result=db.prepare('INSERT INTO orders(user_id, product_id, quantity, total)
VALUES(?, ?, ?, ?)').run(user, product_id, quantity, p.price*quantity);
          db.exec('COMMIT'); return send(201, db.prepare('SELECT * FROM orders WHERE
id=?').get(result.lastInsertRowid));
        } catch(e) {db.exec('ROLLBACK'); throw e;}
      }
      send(404, {error: 'Not found'});
    } catch(e)
{send(e.message==='Unauthorized'?401:500, {error:e.message==='Unauthorized'? 'Unauthorized': 'Internal
server error'});}
  });
  server.on('close', ()=>db.close()); return server;
}
if(process.argv[1]===fileURLToPath(import.meta.url)) {
  const demo=process.env.DEMO_MODE==='1';
  if(!demo && (!process.env.TELEGRAM_BOT_TOKEN || !process.env.BACKEND_API_KEY)) throw Error('Set
TELEGRAM_BOT_TOKEN and BACKEND_API_KEY, or DEMO_MODE=1 for local testing');
```

```
    createApp({demo,dbPath:process.env.DB_PATH||fileURLToPath(new URL('./data/shop.sqlite',import.meta.url)
    )}).listen(Number(process.env.PORT||3000),process.env.HOST||'127.0.0.1',()=>console.log('Shop backend
    started'));
}
```

backend/index.html

```
<!doctype html><html lang="en"><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1"><title>Campus Shop</title>
<script src="https://telegram.org/js/telegram-web-app.js"></script>
<style>body{font:16px system-ui;background:#eef2f7;color:#17233b;margin:0}main{max-
width:650px;margin:40px auto;padding:24px}article{background:white;border-
radius:16px;padding:22px;margin:16px 0;box-shadow:0 5px 20px
#17233b10}button{background:#2563eb;color:white;border:0;border-
radius:8px;padding:12px;cursor:pointer}input{width:65px;padding:10px;margin:0
12px}small{color:#637089}#status{padding:12px;white-space:pre-wrap}h1{font-size:36px}</style>
<main><small>LAB 3 · TELEGRAM MINI APP</small><h1>Campus Shop</h1><p>Study essentials. Order in a few
taps.</p><div id="products"></div><button id="history">My orders</button><p id="status"
role="status"></p></main>
<script>
const tg=window.Telegram?.WebApp;tg?.ready();tg?.expand();
const status=document.querySelector('#status');
async function api(path,method='GET',body){const r=await fetch(path,{method,headers:{'Content-
Type':'application/json','X-Telegram-Init-
Data':tg?.initData||''},body:body?JSON.stringify(body):undefined});const data=await
r.json();if(!r.ok)throw Error(data.error);return data;}
const money=c=>`€${(c/100).toFixed(2)}`;
async function load(){const products=await api('/api/products');const
root=document.querySelector('#products');root.replaceChildren();for(const p of products){const
card=document.createElement('article');const
title=document.createElement('h2');title.textContent=p.name;const
info=document.createElement('p');info.textContent=`${money(p.price)} · ${p.stock} in stock`;const qty=doc
ument.createElement('input');qty.type='number';qty.min=1;qty.max=Math.min(100,p.stock);qty.value=1;qty.se
tAttribute('aria-label','Quantity for ${p.name}');const buy=document.createElement('button');buy.textCont
ent='Order';buy.disabled=p.stock===0;buy.onclick=async()=>{buy.disabled=true;try{const o=await
api('/api/orders','POST',{product_id:p.id,quantity:Number(qty.value)});status.textContent=`Order #${o.id}
confirmed · ${money(o.total)}`;tg?.HapticFeedback?.notificationOccurred('success');await load();}catch(e
){status.textContent=e.message;buy.disabled=false;}};card.append(title,info,qty,buy);root.append(card);}}
document.querySelector('#history').onclick=async()=>{try{const orders=await
api('/api/orders');status.textContent=orders.length?orders.map(o=>`#${o.id} · ${o.quantity} item(s) ·
${money(o.total)}`).join('\n`):'No orders yet';}catch(e){status.textContent=e.message;}};
load().catch(e=>status.textContent=e.message);
</script></html>
```

backend/server.test.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { createHmac } from 'node:crypto';
import { createApp } from './server.js';

test('catalog, authenticated orders, stock, history and Mini App signature', async()=>{
  const app=createApp({dbPath:'memory:',apiKey:'test-key',token:'test-token'});
  await new Promise(r=>app.listen(0, '127.0.0.1', r));
  const base=`http://127.0.0.1:${app.address().port}`;
  const headers={'Content-Type':'application/json','X-Api-Key':'test-key'};
  const
  order=(data,h=headers)=>fetch(base+'/api/orders',{method:'POST',headers:h,body:JSON.stringify(data)});
  try {
    assert.equal((await fetch(base+'/health')).status,200);
    assert.match(await (await fetch(base+'/')).text(),/Campus Shop/);
    assert.equal((await (await fetch(base+'/api/products')).json()).length,3);
    assert.equal((await order({user_id:123,product_id:1,quantity:2},{})).status,401);
    const first=await
  order({user_id:123,product_id:1,quantity:2});assert.equal(first.status,201);assert.equal((await
  first.json()).total,1000);
    assert.equal((await order({user_id:123,product_id:1,quantity:0})).status,400);
    assert.equal((await order({user_id:123,product_id:999,quantity:1})).status,404);
    assert.equal((await order({user_id:123,product_id:1,quantity:19})).status,409);
    assert.equal((await (await fetch(base+'/api/products')).json())[0].stock,18);
    assert.equal((await (await fetch(base+'/api/orders?user_id=123',{headers})).json()).length,1);
    assert.equal((await (await fetch(base+'/api/orders?user_id=456',{headers})).json()).length,0);
    const p=new
  URLSearchParams({auth_date:String(Math.floor(Date.now()/1000)),user:JSON.stringify({id:456})});
    const secret=createHmac('sha256','WebAppData').update('test-token').digest();
    const check=[...p.entries()].sort(([a],[b])=>a.localeCompare(b)).map(([k,v])=>`${k}=${v}`).join('\n');
    p.set('hash',createHmac('sha256',secret).update(check).digest('hex'));
    const signed={'Content-Type':'application/json','X-Telegram-Init-Data':p.toString()};
    assert.equal((await order({user_id:999,product_id:2,quantity:1},signed)).status,201);
    const history=await (await
  fetch(base+'/api/orders',{headers:signed})).json();assert.equal(history[0].user_id,'456');
    signed['X-Telegram-Init-Data']+='tampered';assert.equal((await
  fetch(base+'/api/orders',{headers:signed})).status,401);
    }finally{await new Promise(r=>app.close(r));}
});
```

bot/main.py

```
"""Telegram long-polling bot; Python standard library only."""
import io
import json
import os
import time
import urllib.error
import urllib.parse
import urllib.request
import uuid

TOKEN = os.environ.get('TELEGRAM_BOT_TOKEN', '')
BACKEND = os.environ.get('BACKEND_URL', 'http://127.0.0.1:3000').rstrip('/')
KEY = os.environ.get('BACKEND_API_KEY', '')
WEB_APP = os.environ.get('WEB_APP_URL', '')

def request(url, data=None, headers=None):
    req = urllib.request.Request(url, data=data, headers=headers or {})
    try:
        with urllib.request.urlopen(req, timeout=40) as response:
            return json.load(response)
    except urllib.error.HTTPError as exc:
        try:
            message = json.load(exc).get('error', 'Request failed')
        except (ValueError, AttributeError):
            message = 'Request failed'
        raise RuntimeError(message) from None

def telegram(method, **params):
    result = request(f'https://api.telegram.org/bot{TOKEN}/{method}',
                    json.dumps(params).encode(), {'Content-Type': 'application/json'})
    if not result.get('ok'):
        raise RuntimeError('Telegram request failed')
    return result['result']

def backend(path, data=None):
    return request(BACKEND + path, json.dumps(data).encode() if data is not None else None,
                  {'Content-Type': 'application/json', 'X-API-Key': KEY})

def send(chat, text, buttons=None):
    params = {'chat_id': chat, 'text': text}
    if buttons:
        params['reply_markup'] = {'inline_keyboard': buttons}
    telegram('sendMessage', **params)

def sticker(chat):
    # Upload an original static WEBP sticker rather than relying on a third-party file ID.
    from pathlib import Path
    content = Path(__file__).with_name('welcome.webp').read_bytes()
    boundary = uuid.uuid4().hex
    body = (f'--{boundary}\r\nContent-Disposition: form-data; name="chat_id"\r\n\r\n{chat}\r\n'
           f'--{boundary}\r\nContent-Disposition: form-data; name="sticker";'
           f'--{boundary}\r\n\r\n'
           'Content-Type: image/webp\r\n\r\n').encode() + content + f'\r\n--{boundary}--\r\n'.encode()
    filename = "welcome.webp\r\n"
    result = request(f'https://api.telegram.org/bot{TOKEN}/sendSticker', body,
                    {'Content-Type': f'multipart/form-data; boundary={boundary}'})
    if not result.get('ok'):
        raise RuntimeError('Sticker upload failed')

def catalog(chat):
    products = backend('/api/products')
    buttons = [[{'text': f'{p["name"]} · €{p["price"]/100:.2f} ({p["stock"]} left)',
                  'callback_data': f'choose:{p["id"]}'}] for p in products if p['stock'] > 0]
    send(chat, 'Choose a product:', buttons)

def handle(update):
    if 'callback_query' in update:
        query = update['callback_query']
        telegram('answerCallbackQuery', callback_query_id=query['id'])
        chat = query.get('message', {}).get('chat', {}).get('id')
        if chat is None:
            return
        user = query['from']['id']
        data = query.get('data', '')
        if data == 'catalog':
            catalog(chat)
        elif data == 'orders':
            orders = backend('/api/orders?' + urllib.parse.urlencode({'user_id': user}))
            send(chat, '\n'.join(f'{o["id"]} · {o["quantity"]} item(s) · €{o["total"]/100:.2f}' for o in
                                orders) or 'No orders yet.')
        elif data.startswith('choose:'):
            product = int(data.split(':')[1])
            send(chat, 'Confirm purchase of one item?', [{'text': 'Confirm order', 'callback_data':
                f'buy:{product}'}, {'text': 'Cancel', 'callback_data': 'catalog'}])
```

```

        elif data.startswith('buy:'):
            order = backend('/api/orders', {'user_id': user, 'product_id': int(data.split(':')[1]),
            'quantity': 1})
            # Remove the confirmation keyboard to prevent ordinary double taps.
            telegram('editMessageReplyMarkup', chat_id=chat, message_id=query['message']['message_id'],
            reply_markup={'inline_keyboard': []})
            send(chat, f"Order #{order['id']} confirmed! Total: €{order['total']/100:.2f}")
        elif 'message' in update:
            message = update['message']
            chat = message['chat']['id']
            text = message.get('text', '').split('@')[0]
            if text == '/catalog':
                catalog(chat)
            else:
                buttons = [{ 'text': 'Catalog', 'callback_data': 'catalog' }, { 'text': 'My orders',
            'callback_data': 'orders' }]
                if WEB_APP and message['chat']['type'] == 'private':
                    buttons.append([{'text': 'Open Mini App', 'web_app': {'url': WEB_APP}}])
                send(chat, 'Welcome to Campus Shop! Browse products, confirm an order, and view your
            history.', buttons)
                if text == '/start':
                    sticker(chat)

def main():
    if not TOKEN or not KEY:
        raise SystemExit('Set TELEGRAM_BOT_TOKEN and BACKEND_API_KEY before starting the bot.')
    if WEB_APP and not WEB_APP.startswith('https://'):
        raise SystemExit('WEB_APP_URL must use HTTPS.')
    backend('/health')
    telegram('getMe')
    telegram('deleteWebhook', drop_pending_updates=False)
    offset = 0
    while True:
        try:
            updates = telegram('getUpdates', offset=offset, timeout=25, allowed_updates=['message',
            'callback_query'])
            for update in updates:
                try:
                    handle(update)
                except (RuntimeError, ValueError, KeyError, urllib.error.URLError):
                    chat = update.get('message', update.get('callback_query', {})).get('message',
            {})).get('chat', {}).get('id')
                    if chat:
                        send(chat, 'Unable to complete the request. Check stock and try again.')
                        offset = update['update_id'] + 1
                except (RuntimeError, urllib.error.URLError):
                    print('Connection error; retrying in 5 seconds.', flush=True)
                    time.sleep(5)

if __name__ == '__main__':
    main()

```

bot/test_bot.py

```
import unittest
from unittest.mock import patch
import main

class BotTests(unittest.TestCase):
    @patch.object(main, 'telegram')
    @patch.object(main, 'backend')
    def test_purchase(self, backend, telegram):
        backend.return_value = {'id': 7, 'total': 500}
        main.handle({'callback_query': {'id': 'q1', 'from': {'id': 123}, 'data': 'buy:1', 'message':
{'message_id': 10, 'chat': {'id': 123}}}})
        backend.assert_called_once_with('/api/orders', {'user_id': 123, 'product_id': 1, 'quantity': 1})
        self.assertEqual(telegram.call_args.kwargs['text'], 'Order #7 confirmed! Total: €5.00')

    @patch.object(main, 'sticker')
    @patch.object(main, 'telegram')
    @patch.object(main, 'WEB_APP', 'https://example.com')
    def test_start(self, telegram, sticker):
        main.handle({'message': {'text': '/start', 'chat': {'id': 123, 'type': 'private'}}})
        sticker.assert_called_once_with(123)
        keyboard = telegram.call_args.kwargs['reply_markup']['inline_keyboard']
        self.assertEqual(keyboard[-1][0]['web_app']['url'], 'https://example.com')

if __name__ == '__main__':
    unittest.main()
```

.env.example

```
# Copy to .env and replace locally. Never commit the real .env.  
TELEGRAM_BOT_TOKEN=  
BACKEND_API_KEY=  
BACKEND_URL=http://127.0.0.1:3000  
WEB_APP_URL=  
HOST=127.0.0.1  
PORT=3000
```

.gitignore

```
.env  
.venv/  
__pycache__/  
backend/data/  
*.log
```

README.md

Lab 3 – Campus Shop

Telegram shopping bot (Python), back-end business logic (Node.js + SQLite), and Telegram Mini App. Implements the advanced task in the supplied Lab 3 description.

Requirements

Node.js 24+ and Python 3.12+. Runtime code uses only standard libraries; no package installation is required. SQLite support is built into Node.js. The bundled original sticker needs no image library at runtime.

Local demo

```
```sh
cd /workspace/lab3/backend
DEMO_MODE=1 node server.js
```
```

Open port 3000 locally. Demo mode assigns requests to a fixed test user; **never enable it on a public deployment**. SQLite is stored in `backend/data/shop.sqlite`. Demo purchases persist. Use `DB\_PATH=/tmp/lab3-demo.sqlite` for a separate demo database.

Telegram setup

1. In Telegram, open the official `@BotFather`, run `/newbot`, choose a name and a unique username ending in `bot`, and obtain the token. Never include it in the report or Git.
2. Copy `.env.example` to `.env`. Set `TELEGRAM\_BOT\_TOKEN` and a strong random `BACKEND\_API\_KEY` (for example, generate one with `python -c 'import secrets; print(secrets.token\_hex(32))'`). The bot and backend must share these values.
3. Deploy the backend through an HTTPS reverse proxy or a trusted HTTPS development tunnel. Set `WEB\_APP\_URL` to its public HTTPS root. The proxy must forward the `X-Telegram-Init-Data` header. Keep the backend bound to loopback when the reverse proxy runs on the same host; set `HOST=0.0.0.0` only when required by the hosting platform. Restrict access and protect the API key.
4. From the repository root, load variables in each terminal and start the backend and bot:

```
```sh
set -a
. ./env
set +a
Terminal 1 (DEMO_MODE must be unset for real Telegram use):
cd backend
node server.js
Terminal 2, after loading .env from repository root:
python bot/main.py
```
```

5. Open the bot in a private chat and send `/start`. Verify the sticker, Catalog, confirmation, My orders, and Open Mini App. The Mini App keyboard is only offered in private chats. A menu button is optional; set it through BotFather if desired.

The bot uses long polling and removes an existing webhook when started. Run only one bot instance per token. Telegram traffic requires `api.telegram.org`; loading Telegram's Mini App SDK requires `telegram.org`. The Mini App must be accessed inside Telegram for authenticated real orders; an ordinary browser is only usable with local demo mode.

Validation

```
```sh
cd backend && npm test
cd ../bot && python -m unittest -v
```
```

The backend integration test exercises HTTP routing, order totals, stock updates, rejection of invalid quantities/insufficient stock, user isolation, valid Mini App authentication, and tampering. Two Python tests use mocks to check the welcome flow and ordering integration. These mock tests do not prove a live Telegram connection.

Structure

- `bot/main.py`: Python Bot API client, inline keyboards, sticker upload, order flow.
 - `bot/welcome.webp`: original sticker.
 - `backend/server.js`: HTTP API, Telegram signature validation, SQLite transactions.
 - `backend/index.html`: Mini App catalog and order history.
 - `report/REPORT.md`, `report/Lab3-report.pdf`, `report/Lab3-report.docx`, and `report/Lab3-report.html`: report for Matvei Krautsou with four genuine mobile-browser screenshots and full source appendices.
- `Lab3-Matvei-Krautsou.zip` contains the complete submission.

Limitations

This is a laboratory prototype: no payments, shipping, stock administration, or production deployment automation. Orders are independent purchases; API retries do not have idempotency keys. A repeated confirmation may create another order. Live Telegram testing and Telegram screenshots require the student's BotFather token and HTTPS deployment. The bot token was verified with Telegram and the student supplied screenshots demonstrating live purchases and history. The report documents those observations. Original Telegram screenshots still need uploading as files for embedding; inline chat images were not available to the report builder.

Rebuild the submission report

Install optional report tools with `python -m pip install reportlab python-docx Pillow`, then run `python

report/build_report.py`. The generator builds PDF, DOCX, standalone HTML, and a secret-free ZIP. Mini App images are actual local Chromium captures. They do not imply a public HTTPS Mini App deployment.

report/build_report.py

```
"""Build reports and a secret-free submission archive. Run from any directory.
Requires: python -m pip install reportlab python-docx Pillow
"""
from pathlib import Path
from xml.sax.saxutils import escape
import base64
import html
import re
import zipfile
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Image, PageBreak, Preformatted
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER
from reportlab.lib import colors
from docx import Document
from docx.shared import Inches, Pt

HERE = Path(__file__).resolve().parent
ROOT = HERE.parent
styles = getSampleStyleSheet()
styles.add(ParagraphStyle('CodeSmall', fontName='Courier', fontSize=6.5, leading=8))
story = []
doc = Document()
doc.core_properties.author = 'Matvei Krautsou'
doc.core_properties.title = 'Lab 3 - Campus Shop'
parts = ['<!doctype html><meta charset="utf-8"><title>Matvei Krautsou - Lab 3</title><style>body{font:16px Arial;max-width:900px;margin:40px auto;padding:20px;line-height:1.5}img{display:block;max-width:100%;max-height:760px;margin:auto}pre{font:12px monospace;white-space:pre-wrap;background:#f1f4f8;padding:16px}h1,h2,h3{color:#193c6a}@media print{img{max-height:650px}h2{break-after:avoid}}</style>']
for line in (HERE / 'REPORT.md').read_text().splitlines():
    if line.startswith('!['):
        name = re.search(r'\\[\\((.*)\\)', line).group(1)
        path = HERE / name
        story.append(Image(str(path), width=260, height=520, kind='proportional'))
        doc.add_picture(str(path), width=Inches(3.2))
        data = base64.b64encode(path.read_bytes()).decode()
        parts.append(f'')
    elif line.startswith('#'):
        level = len(line) - len(line.lstrip('#'))
        text = line.lstrip('# ')
        story.append(Paragraph(escape(text), styles['Title' if level == 1 else 'Heading2']))
        doc.add_heading(text, level=min(level, 3))
        n = min(level, 3)
        parts.append(f'<h{n}>{html.escape(text)}</h{n}>')
    elif line:
        story.append(Paragraph(escape(line), styles['BodyText']))
        doc.add_paragraph(line)
        parts.append(f'<p>{html.escape(line)}</p>')
    else:
        story.append(Spacer(1, 8))

sources = ['backend/package.json', 'backend/server.js', 'backend/index.html',
          'backend/server.test.js', 'bot/main.py', 'bot/test_bot.py',
          '.env.example', '.gitignore', 'README.md', 'report/build_report.py']
for name in sources:
    text = (ROOT / name).read_text()
    if re.search(r'\d{8,12}:[A-Za-z0-9_-]{30,}', text):
        raise RuntimeError('Token-like content detected in ' + name)
    story.extend([PageBreak(), Paragraph(escape(name), styles['Heading2'])])
    # Preserve every source character, wrapping only the printed representation.
    import textwrap
    printed = '\n'.join(piece for line in text.expandtabs(4).splitlines()
                        for piece in (textwrap.wrap(line, width=105, replace_whitespace=False,
                                                    drop_whitespace=False) or ['']))
    story.append(Preformatted(printed, styles['CodeSmall']))
    doc.add_page_break()
    doc.add_heading(name, level=2)
    paragraph = doc.add_paragraph()
    run = paragraph.add_run(text)
    run.font.name = 'Courier New'
    run.font.size = Pt(7)
    parts.append(f'<h2>{html.escape(name)}</h2><pre>{html.escape(text)}</pre>')

def footer(canvas, document):
    canvas.setFont('Helvetica', 8)
    canvas.setFillColors(colors.HexColor('#667085'))
    canvas.drawString(42, 24, 'Matvei Krautsou | RPA Lab 3 | Campus Shop')
    canvas.drawRightString(553, 24, str(document.page))

SimpleDocTemplate(str(HERE / 'Lab3-report.pdf'), title='Lab 3 - Campus Shop',
                  author='Matvei Krautsou', leftMargin=42, rightMargin=42,
                  topMargin=42, bottomMargin=42).build(story, onFirstPage=footer, onLaterPages=footer)
doc.save(HERE / 'Lab3-report.docx')
(HERE / 'Lab3-report.html').write_text('\n'.join(parts))
archive = ROOT / 'Lab3-Matvei-Krautsou.zip'
with zipfile.ZipFile(archive, 'w', zipfile.ZIP_DEFLATED) as z:
    for path in sorted(ROOT.rglob('*')):
        relative = path.relative_to(ROOT)
        if (not path.is_file() or path == archive or path.suffix == '.zip'
            or path.name == '.env' or any(p in {'git', '__pycache__', '.venv', 'data'} for p in
```

```
relative.parts)):
    continue
    z.write(path, Path('lab3') / relative)
print('Generated PDF, DOCX, standalone HTML and secret-free submission ZIP')
```